Software Requirements Specification (SRS)

Restora – Operational Intelligence Platform for Restaurants

Prepared for: Mobile Computing (CCS3102), Group 32 **Document Version:** 2.0 (Consolidated & Enhanced) **Date:** July 08, 2026 **Prepared by:** Nanthakumar Vepusanan (FC115287 / FC221023), Baskaran Natheesan (FC115266 / FC221014), K.D.S. Yashoda (FC115290 / FC221024) **Standard Followed:** ISO/IEC/IEEE 29148:2018 (with IEEE 830 structural conventions)

Revision Note

This version consolidates the original **Group Project Proposal** and the **v1.0 SRS** into a single authoritative specification. Gaps identified between the two source documents (e.g., the Proposal's lighter functional list vs. the SRS's FR1–FR42 detail) have been resolved in favor of the more precise SRS wording, and additional production-grade requirements have been added where the source documents were silent. All newly introduced content is marked **[NEW]**. All assumptions made to resolve ambiguity are marked **[ASSUMPTION]**.

---

1. Introduction

1.1 Purpose

This SRS defines the complete functional and non-functional requirements for **Restora**, a mobile-first Android application supporting ingredient inventory management, expiry monitoring, waste tracking, cost/expense tracking, and operational analytics for small and medium restaurant businesses. It is intended for the development team (Group 32), the CCS3102 course evaluators, and any future maintainer. It is written to a level of detail sufficient for direct implementation, quality assurance test-case derivation, and project management planning (WBS/effort estimation), without requiring further clarification of scope or intended behavior.

1.2 Scope

Restora is a cross-platform mobile application (React Native/TypeScript, targeting Android 8.0+) backed by Firebase (Firestore, Authentication, Cloud Messaging, Cloud Functions) and integrated with the Google Gemini API for natural-language insight generation. The system supports two roles — **Admin** and **Staff** — operating within a single restaurant context identified by a unique restaurant code.

In scope:

- Role-based authentication, registration, and staff onboarding with Admin approval
- Batch-based ingredient inventory tracking with FIFO ordering and expiry status indicators
- Automated expiry monitoring with push notification alerts
- Waste logging with automatic cost-loss calculation and historical records
- Cost and expense tracking at ingredient and inventory level
- An Admin-only analytics dashboard summarizing waste, cost, and inventory trends
- A read-only, role-filtered AI assistant answering natural-language queries
- Real-time synchronization across all signed-in devices
- **[NEW]** Audit logging and activity history
- **[NEW]** User profile and restaurant settings management
- **[NEW]** Data export/reporting (CSV/PDF)
- **[NEW]** Basic offline-tolerant operation with local caching

Out of scope: point-of-sale integration, supplier ordering/procurement automation, payroll, multi-restaurant management under a single account, full offline-first conflict resolution beyond Firestore's default caching, and non-English localization.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
FIFO	First-In, First-Out — the principle that older stock should be consumed before newer stock
FCM	Firebase Cloud Messaging — push notification delivery service
FR	Functional Requirement
NFR	Non-Functional Requirement
RBAC	Role-Based Access Control
Batch	A single recorded receipt of an ingredient (quantity, cost, supplier, dates)
Admin	Restaurant owner/manager role with full system access
Staff	Operational employee role with limited, non-financial access
Restaurant Code	Unique code required for Staff self-registration

SLA	Service Level Agreement
MVP	Minimum Viable Product
TLS	Transport Layer Security
PII	Personally Identifiable Information

1.4 References

- Restora – Group Project Proposal, Mobile Computing (CCS3102), Group 32
- Restora SRS v1.0, Group 32, June 25, 2026
- Firebase Documentation (Firestore, Authentication, Cloud Messaging, Cloud Functions)
- Google Gemini API Documentation
- ISO/IEC/IEEE 29148:2018 — Systems and software engineering — Life cycle processes — Requirements engineering

1.5 Overview

Section 2 describes overall product context, functions, user classes, environment, constraints, and assumptions. Section 3 specifies functional requirements grouped into modules with full use-case-style detail. Section 4 specifies non-functional requirements. Section 5 defines user roles and permissions. Section 6 provides key use cases. Section 7 lists business rules. Section 8 defines the data model. Section 9 specifies external interfaces. Sections 10–12 cover system constraints, risks/assumptions, and future enhancements.

---

2. Overall Description

2.1 Product Perspective

Restora is a new, standalone mobile application; it does not replace or interface with any existing enterprise restaurant management system. It is a self-contained client-server system in which the React Native mobile client communicates directly with Firebase backend services. There is no custom backend server; Firestore is the system of record, Firebase Authentication manages identity, and FCM delivers push notifications. The Gemini API is invoked via a thin Cloud Function proxy **[ASSUMPTION: a server-side proxy, not a direct client call, is used to keep the Gemini API key off-device and enable role-based context filtering server-side]**.

2.2 Product Functions (Summary)

1. Account registration, login, and role-based routing
2. Staff onboarding with Admin approval workflow
3. Batch-based inventory CRUD with FIFO ordering and expiry status
4. Automated expiry monitoring and push notification alerts
5. Waste event logging with automatic cost-loss calculation
6. Cost/expense tracking and inventory valuation
7. Admin analytics dashboard (charts, trends, rankings)
8. AI assistant for natural-language inventory/cost/waste queries
9. Real-time multi-device data synchronization
10. **[NEW]** Audit trail of all create/update/delete/approve/reject actions
11. **[NEW]** Profile and restaurant configuration management
12. **[NEW]** Report export (CSV/PDF) of inventory, waste, and cost data
13. **[NEW]** In-app notification inbox with read/unread state

2.3 User Classes and Characteristics

2.3.1 Admin

The restaurant owner/manager who creates the restaurant account, receives a system-generated restaurant code, approves/rejects staff registration requests, and has unrestricted access to inventory, waste logs, cost data, the analytics dashboard, and the AI Assistant. Assumed basic smartphone literacy, no technical training.

2.3.2 Staff

Kitchen/floor employees who register with a restaurant code and, once approved, gain operational access limited to inventory viewing, batch registration, expiry lookups, waste logging, and push notifications. Cannot view financial summaries, cost breakdowns, or the analytics dashboard, and cannot manage other accounts.

2.3.3 [NEW] Super Admin / System Maintainer (non-UI role)

A development-team-only role with access to Firebase Console and Cloud Function logs for maintenance, monitoring, and incident response. Not exposed within the mobile application UI. **[ASSUMPTION]** included because a production system needs an operational owner distinct from restaurant Admins.

2.4 Operating Environment

- **Client platform:** Android 8.0 (API level 26) and above; minimum 2 GB RAM assumed
- **Mobile framework:** React Native (JavaScript/TypeScript)
- **Backend:** Firebase Firestore, Firebase Authentication, Firebase Cloud Messaging, Firebase Cloud Functions

- **AI service:** Google Gemini API, accessed over HTTPS via Cloud Function proxy
- **Network:** Requires active internet connectivity (Wi-Fi/mobile data) for real-time sync; brief connectivity loss tolerated via Firestore local cache and automatic re-sync

2.5 Design and Implementation Constraints

- Mobile-first, Android-only client; no desktop client in scope
- Firebase-only backend architecture; no alternative backend permitted within project scope
- Must support real-time synchronization across all devices signed into the same restaurant account
- Must remain usable on low-end Android hardware without perceptible lag during core operations
- All requirement statements avoid unmeasurable terms (e.g., "user-friendly") unless accompanied by a measurable definition
- Development tools: Visual Studio Code, Android Studio, Figma (free plan), GitHub (free plan)
- Charting: Victory Native
- **[NEW]** All monetary calculations must use fixed-point/decimal-safe arithmetic (not raw floating point) to avoid rounding errors in financial figures

2.6 Assumptions and Dependencies

- Each physical restaurant is represented by exactly one restaurant account and one restaurant code at any given time
- Staff possess a valid restaurant code, obtained directly from their Admin, before attempting registration
- The Gemini API free tier provides sufficient request quota for the project's expected usage volume during development and demonstration
- Devices used to run the application have Google Play Services installed (required for FCM)
- Monetary values are recorded and displayed in a single currency configured at the restaurant level; multi-currency support is not required
- **[ASSUMPTION]** Only one Admin per restaurant is supported in the MVP; multi-admin support is a future enhancement
- **[ASSUMPTION]** The system does not require App Store/Play Store publication for course submission; sideloaded APK or Expo build is sufficient
- **[ASSUMPTION]** No payment processing is required; the app is free to use for this academic scope

---

3. Functional Requirements

Requirement IDs follow the pattern **FR-XXX**. Each requirement lists Description, Priority, Preconditions, Main Flow, Alternate Flow, Postconditions, and Acceptance Criteria.

3.1 Module: Authentication & Access Control

FR-001 — Admin Registration

- **Description:** The system shall allow a new Admin to register a restaurant account using an email address and password, generating a unique, system-wide restaurant code on success.
- **Priority:** High
- **Preconditions:** User has no existing account with the given email.
- **Main Flow:** User enters email/password → submits → system creates Firebase Auth user, creates Restaurant document, generates unique restaurantCode → user is signed in and routed to Admin Dashboard.
- **Alternate Flow:** Email already registered → system displays "Account already exists" error and offers login link.
- **Postconditions:** Restaurant and Admin User documents exist in Firestore; restaurant code displayed to Admin.
- **Acceptance Criteria:** Given a unique email and valid password (≥8 chars), when registration is submitted, then a Restaurant document and Admin User document are created and a restaurant code is shown within 3 seconds.

FR-002 — Staff Registration

- **Description:** The system shall allow a prospective Staff user to register using email, password, and a restaurant code, rejecting registration if the code does not match an existing restaurant.
- **Priority:** High
- **Preconditions:** A valid restaurant code exists.
- **Main Flow:** Staff enters email/password/restaurant code → system validates code against Restaurant collection → creates User document with status "Pending".
- **Alternate Flow:** Invalid code → system rejects with "Invalid restaurant code" error, no account created.
- **Postconditions:** Pending User document created, linked to restaurantId.
- **Acceptance Criteria:** Given an invalid restaurant code, when submitted, then no User document is created and an inline error is shown.

FR-003 — Pending Account Restriction

- **Description:** The system shall set new Staff accounts to status "Pending" and prevent Pending accounts from accessing inventory, waste, or cost data.
- **Priority:** High
- **Preconditions:** Staff account created via FR-002.
- **Main Flow:** Pending Staff logs in → routed to Pending Approval screen → all data routes blocked client- and server-side.
- **Alternate Flow:** Pending Staff attempts direct Firestore read → denied by Security Rules.

- **Postconditions:** No data exposure to Pending accounts.
- **Acceptance Criteria:** A Pending account's Firestore read request for inventory/waste/cost collections is denied (403/permission-denied) 100% of the time.

FR-004 — Staff Approval/Rejection

- **Description:** The system shall allow an authenticated Admin to view all Pending staff accounts for their restaurant and approve/reject each individually.
- **Priority:** High
- **Preconditions:** Admin authenticated; at least one Pending account exists.
- **Main Flow:** Admin opens Staff Management → views Pending list → taps Approve/Reject → status updated.
- **Alternate Flow:** Admin rejects → account status set to "Rejected"; user notified and denied further access.
- **Postconditions:** Status field updated to Approved or Rejected.
- **Acceptance Criteria:** Status change is reflected in the Staff user's session state within 5 seconds (real-time listener).

FR-005 — Immediate Access on Approval

- **Description:** The system shall grant an approved Staff account access to operational features immediately upon approval, without re-registration.
- **Priority:** High
- **Main Flow:** Admin approves → Staff's active/next session detects status change via real-time listener → routed to Inventory List.
- **Postconditions:** Staff gains operational route access.
- **Acceptance Criteria:** No re-login required for access to be granted (verified via listener-driven UI state update).

FR-006 — Staff Deactivation

- **Description:** The system shall allow an Admin to deactivate/remove a Staff account, revoking access on all signed-in devices within 60 seconds, enforced via Firestore Security Rules.
- **Priority:** High
- **Main Flow:** Admin selects Staff → Deactivate → confirmation dialog → status set to "Deactivated" → Security Rules re-evaluate on next request.
- **Alternate Flow:** Admin cancels confirmation → no change.
- **Postconditions:** Deactivated account denied all reads/writes; FCM token removed (see FR-042).
- **Acceptance Criteria:** A deactivated account's subsequent Firestore request is denied within 60 seconds of deactivation.

FR-007 — Login Authentication

- **Description:** The system shall authenticate all login attempts using Firebase Authentication (email/password) and deny unauthenticated access to any inventory, waste, cost, or analytics endpoint.
- **Priority:** High

- **Acceptance Criteria:** Any Firestore request without a valid auth token is denied by Security Rules.

FR-008 — Role-Based Data Access Enforcement

- **Description:** The system shall enforce RBAC such that a Staff account is structurally prevented (via Firestore Security Rules, not only UI hiding) from reading/writing financial summary data, cost breakdowns, or analytics aggregates.
- **Priority:** High
- **Acceptance Criteria:** Direct Firestore SDK calls from a Staff-authenticated context to cost/analytics collections return permission-denied in 100% of test cases.

FR-009 — [NEW] Password Reset

- **Description:** The system shall allow a user to request a password reset via a verified email link.
- **Priority:** Medium
- **Main Flow:** User taps "Forgot Password" → enters email → Firebase sends reset link → user sets new password.
- **Acceptance Criteria:** Reset link expires after 1 hour; used links are invalidated after one use.

FR-010 — [NEW] Session Management

- **Description:** The system shall maintain a persistent authenticated session using Firebase Auth tokens, auto-refreshing before expiry, and shall log the user out if the refresh token is revoked (e.g., on deactivation).
- **Priority:** Medium
- **Acceptance Criteria:** Session remains valid across app restarts until explicit logout or revocation; token refresh occurs transparently to the user.

3.2 Module: Inventory Management

FR-011 — Create Batch

- **Description:** The system shall allow an authenticated user to create a new inventory batch recording: ingredient name, quantity, unit of measure, unit cost, supplier, date received, and expiry date.
- **Priority:** High
- **Main Flow:** User opens Batch Registration form → fills fields → submits → validated (FR-012) → Firestore document created.
- **Postconditions:** New Inventory Batch document persisted, visible to all devices within 5 seconds (FR-041).
- **Acceptance Criteria:** A valid submission creates exactly one batch document with all required fields populated.

FR-012 — Batch Field Validation

- **Description:** The system shall reject batch creation if: ingredient name is missing, quantity ≤ 0, unit cost < 0, or expiry date is before date received.
- **Priority:** High
- **Acceptance Criteria:** Each invalid condition individually blocks submission with a field-specific error message.

FR-013 — FIFO Grouping and Ordering

- **Description:** The system shall group batches by ingredient name in the inventory list and order batches within each group by date received ascending (oldest first).
- **Priority:** High
- **Acceptance Criteria:** For any ingredient group with ≥2 batches, the batch with the earliest dateReceived always appears first.

FR-014 — FIFO Indicator

- **Description:** The system shall display a FIFO indicator on the oldest unconsumed batch within each ingredient group, visually distinguishing it from later batches.
- **Priority:** Medium
- **Acceptance Criteria:** Exactly one FIFO-indicator badge is rendered per ingredient group.

FR-015 — Expiry Status Tag Calculation

- **Description:** The system shall calculate and display an expiry status tag per batch: Green if days-to-expiry > 3; Amber if 0 ≤ days-to-expiry ≤ 3; Red if days-to-expiry < 0.
- **Priority:** High
- **Acceptance Criteria:** Given a batch with a known expiry date, the displayed tag matches the defined thresholds for all boundary values (3, 0, -1 days).

FR-016 — Status Recalculation

- **Description:** The system shall recalculate expiry status for all batches at least once every 24 hours and immediately upon inventory list being opened.
- **Priority:** Medium
- **Acceptance Criteria:** Status tags reflect current date-derived state whenever the list screen is opened, verified via a scheduled Cloud Function run and on-open recompute.

FR-017 — Mark Batch Consumed

- **Description:** The system shall allow marking a batch as fully consumed, removing it from the active inventory list while preserving the record for historical cost/waste reporting.
- **Priority:** Medium
- **Alternate Flow:** User cancels the consume confirmation → no change (see NFR usability requirement on destructive-action confirmation).
- **Acceptance Criteria:** Consumed batches are excluded from active inventory queries but included in historical cost reports.

FR-018 — [NEW] Edit Batch

- **Description:** The system shall allow an authenticated user with appropriate permission to edit a batch's supplier, quantity, or expiry date prior to it being marked consumed, subject to the same validation as FR-012.
- **Priority:** Medium
- **Main Flow:** User selects batch → Edit → modifies fields → saves → validation re-applied → document updated with `lastModifiedBy`/`lastModifiedAt`.
- **Acceptance Criteria:** Edited batch reflects new values immediately across devices; an audit log entry is created (FR-043).

FR-019 — [NEW] Delete/Archive Batch

- **Description:** The system shall allow an Admin to soft-delete (archive) an erroneously created batch, requiring explicit confirmation.
- **Priority:** Low
- **Acceptance Criteria:** Archived batches are hidden from active views but retained in the database for audit purposes (no hard delete).

FR-020 — [NEW] Search, Filter, and Sort Inventory

- **Description:** The system shall allow users to search inventory by ingredient name and filter by status (Green/Amber/Red) and supplier, and sort by expiry date, date received, or ingredient name.
- **Priority:** Medium
- **Acceptance Criteria:** Search results update within 1 second of query input (client-side filter over cached snapshot); filters can be combined.

3.3 Module: Expiry Monitoring

FR-021 — Continuous Expiry Evaluation

- **Description:** The system shall continuously evaluate every active batch's expiry date against the current date and trigger an alert when a batch enters the Amber threshold (days-to-expiry ≤ 3).
- **Priority:** High
- **Acceptance Criteria:** A scheduled Cloud Function run detects and flags all batches crossing the Amber/Red threshold since the last run.

FR-022 — Push Notification Dispatch

- **Description:** The system shall send a push notification via FCM to all devices signed into the affected restaurant account within 5 minutes of a batch entering Amber or Red status.
- **Priority:** High
- **Acceptance Criteria:** Time between threshold crossing (per scheduled check) and notification dispatch is ≤ 5 minutes in ≥95% of cases.

FR-023 — Notification Content

- **Description:** Each expiry push notification shall include ingredient name, quantity, date received, and days remaining/overdue for the triggering batch.
- **Priority:** High
- **Acceptance Criteria:** Notification payload contains all four fields for every dispatched alert.

FR-024 — Duplicate Suppression

- **Description:** The system shall not send a duplicate notification for the same batch and status transition more than once within 24 hours.
- **Priority:** Medium
- **Acceptance Criteria:** No two notifications with the same batchId + status are dispatched within a rolling 24-hour window.

FR-025 — [NEW] Configurable Alert Threshold

- **Description:** The system shall allow an Admin to configure the Amber alert window (default 3 days) per restaurant via Settings.
- **Priority:** Low
- **Acceptance Criteria:** Changing the threshold value affects subsequent status calculations (FR-015) without requiring app redeployment.

3.4 Module: Waste Management

FR-026 — Log Waste Event

- **Description:** The system shall allow a Staff or Admin user to create a waste log entry by selecting an existing batch, specifying quantity wasted, and selecting exactly one reason from: Expired, Burnt, Prep Waste, Leftovers.
- **Priority:** High
- **Acceptance Criteria:** A waste log document is created with all four attributes populated and linked to a valid batchId.

FR-027 — Waste Quantity Validation

- **Description:** The system shall reject a waste log entry where quantity wasted exceeds the remaining recorded quantity of the selected batch.
- **Priority:** High
- **Acceptance Criteria:** Submission is blocked with an inline error when quantityWasted > batch.quantity (remaining).

FR-028 — Automatic Cost-Loss Calculation

- **Description:** The system shall automatically calculate the monetary loss of a waste log entry as (quantity wasted × unit cost recorded on the source batch) and store this value with the record.
- **Priority:** High

- **Acceptance Criteria:** costLoss field equals quantityWasted × batch.unitCost, computed server-side (Cloud Function or security-rule-validated write) to prevent client tampering.

FR-029 — Waste Timestamping

- **Description:** The system shall timestamp every waste log entry with creation date/time and the identifier of the creating user.
- **Priority:** Medium
- **Acceptance Criteria:** Every waste log document has non-null `timestamp` and `loggedBy` fields.

FR-030 — Historical Waste Retention

- **Description:** The system shall retain all waste log entries indefinitely as part of the restaurant's historical waste record, accessible for trend reporting.
- **Priority:** Medium
- **Acceptance Criteria:** No automated deletion process removes waste logs; they remain queryable for any date range.

FR-031 — [NEW] Edit/Void Waste Entry

- **Description:** The system shall allow an Admin to void (not hard-delete) an incorrectly logged waste entry, restoring the corresponding quantity to the source batch and recording a reversal audit entry.
- **Priority:** Low
- **Acceptance Criteria:** Voiding a waste entry increments the batch's remaining quantity and marks the waste log as `voided: true`, excluded from active cost-loss totals.

3.5 Module: Cost and Expense Tracking

FR-032 — Current Inventory Valuation

- **Description:** The system shall calculate total current inventory value as the sum of (remaining quantity × unit cost) across all active, non-expired batches.
- **Priority:** High
- **Acceptance Criteria:** Displayed valuation matches the sum computed independently from raw batch data in test fixtures.

FR-033 — Ingredient Cost by Date Range

- **Description:** The system shall calculate total cost incurred per ingredient over a selectable date range as the sum of (quantity × unit cost) across batches of that ingredient received within the range.
- **Priority:** Medium
- **Acceptance Criteria:** Changing the date range recalculates and updates displayed totals within 2 seconds.

FR-034 — Waste-Loss by Date Range

- **Description:** The system shall calculate total financial loss due to waste over a selectable date range as the sum of monetary loss values of waste log entries created within the range.
- **Priority:** Medium
- **Acceptance Criteria:** Voided waste entries (FR-031) are excluded from this total.

FR-035 — Financial Data Access Restriction

- **Description:** The system shall restrict access to total cost, total inventory value, and waste-loss figures to Admin accounts only, per FR-008.
- **Priority:** High
- **Acceptance Criteria:** Same as FR-008 acceptance criteria, applied to cost-specific queries.

3.6 Module: Analytics Dashboard (Admin Only)

FR-036 — Waste Cost Aggregation

- **Description:** The system shall display waste cost aggregated by day, week, and month on the Admin dashboard, selectable via a date-range control.
- **Priority:** Medium

FR-037 — Top Wasted Ingredients

- **Description:** The system shall display a ranked list of top wasted ingredients by total monetary loss for a selected date range.
- **Priority:** Medium

FR-038 — Cost Breakdown by Ingredient

- **Description:** The system shall display a breakdown of total cost by ingredient for a selected date range.
- **Priority:** Medium

FR-039 — Inventory Valuation on Dashboard

- **Description:** The system shall display current total inventory valuation (per FR-032) on the dashboard home view.
- **Priority:** Medium

FR-040 — Real-Time Dashboard Refresh

- **Description:** The system shall refresh all dashboard figures automatically when underlying inventory, waste, or cost data changes, without manual reload.
- **Priority:** Medium
- **Acceptance Criteria:** Dashboard values update within 5 seconds of an underlying data change (consistent with FR-041 sync SLA).

FR-041 — [NEW] Export Dashboard Report

- **Description:** The system shall allow an Admin to export the current dashboard view (waste, cost, inventory summary) as a CSV or PDF file for a selected date range.
- **Priority:** Low
- **Acceptance Criteria:** Exported file contains all figures visible on screen at time of export, with restaurant name and date range in the header.

3.7 Module: AI Assistant

FR-042 — Natural Language Query Interface

- **Description:** The system shall provide a natural language query interface where an authenticated user submits a text question about inventory, waste, or cost data.
- **Priority:** Medium

FR-043 — Context Construction

- **Description:** The system shall construct a context payload from the restaurant's current Firestore data (relevant subset of inventory, waste log, and cost records) and submit it with the user's query to the Gemini API.
- **Priority:** Medium

FR-044 — Response Display, No Side Effects

- **Description:** The system shall present the Gemini API's response to the user without persisting any modification to inventory, waste, or cost data as a result of an AI Assistant query.
- **Priority:** High
- **Acceptance Criteria:** No Firestore write operations occur as a byproduct of any AI Assistant request in audit logs.

FR-045 — Role-Filtered AI Context

- **Description:** The system shall restrict data included in the AI Assistant's context payload according to the requesting user's role, such that a Staff user's query cannot return financial cost or waste-loss figures restricted under FR-008.
- **Priority:** High
- **Acceptance Criteria:** Given a Staff-originated query asking directly for cost figures, the response contains no monetary values.

FR-046 — AI Error Handling

- **Description:** The system shall display an explicit error message if the Gemini API request fails or times out (after 15 seconds), without crashing the application.
- **Priority:** Medium
- **Acceptance Criteria:** A simulated timeout/failure results in a user-visible error toast/banner and the app remains responsive.

FR-047 — [NEW] AI Query History

- **Description:** The system shall retain the last 20 AI Assistant queries and responses per user for reference within the session, clearable by the user.
- **Priority:** Low

3.8 Module: Notifications System

FR-048 — Device Registration

- **Description:** The system shall register each device for FCM upon successful login and associate the device token with the corresponding user and restaurant account.
- **Priority:** High

FR-049 — Multi-Device Delivery

- **Description:** The system shall deliver expiry alert notifications (per FR-022) to every registered, currently signed-in device for the restaurant, regardless of which device created or last viewed the triggering batch.
- **Priority:** High

FR-050 — Real-Time Data Propagation

- **Description:** The system shall propagate any change to inventory, waste, or cost data to all signed-in devices within 5 seconds under normal network conditions, using Firestore real-time listeners.
- **Priority:** High
- **Acceptance Criteria:** Measured propagation latency ≤5 seconds in ≥95% of trials under standard Wi-Fi conditions.

FR-051 — Token Cleanup

- **Description:** The system shall remove a device's FCM token registration upon user logout or account deactivation, preventing further notification delivery to that device.
- **Priority:** Medium

FR-052 — [NEW] In-App Notification Inbox

- **Description:** The system shall maintain an in-app notification inbox listing recent alerts with read/unread status, independent of whether the push notification was received by the OS.
- **Priority:** Medium
- **Acceptance Criteria:** Notification inbox reflects all Notification documents created for the restaurant in the last 30 days, with unread count badge.

3.9 Module: [NEW] Audit Logging & Activity History

FR-053 — Audit Trail Creation

- **Description:** The system shall create an immutable audit log entry for every create, update, delete/archive, approve, reject, and deactivate action, recording actor, action type, target entity, timestamp, and before/after values where applicable.
- **Priority:** Medium
- **Acceptance Criteria:** Every mutating operation covered by FR-011 through FR-051 produces exactly one corresponding audit log document.

FR-054 — Admin Audit Log View

- **Description:** The system shall allow an Admin to view a filterable, paginated audit log for their restaurant (by user, action type, date range).
- **Priority:** Low

3.10 Module: [NEW] Profile & Settings Management

FR-055 — User Profile View/Edit

- **Description:** The system shall allow a user to view and edit their display name and contact details (not email, which is the login identifier).
- **Priority:** Low

FR-056 — Restaurant Settings

- **Description:** The system shall allow an Admin to configure restaurant name, currency, and the Amber alert threshold (FR-025).
- **Priority:** Medium

FR-057 — Notification Preferences

- **Description:** The system shall allow a user to enable/disable push notifications and choose which alert types (Amber, Red) they receive on their device.
- **Priority:** Low

---

4. Non-Functional Requirements

4.1 Performance

- The inventory list view shall render the first visible batch of up to 200 stored batches within 2 seconds on minimum-spec hardware (Android 8.0, 2 GB RAM) under a 3G-equivalent connection.
- Waste log submission shall confirm locally within 1 second of tapping "Save," independent of network latency, using optimistic local writes with background sync.
- Expiry status recalculation shall complete for a restaurant's full active batch set (up to 1,000 batches) within 3 seconds.

- **[NEW]** Dashboard chart rendering (up to 12 months of aggregated data) shall complete within 3 seconds.

4.2 Security

- All credentials shall be managed exclusively by Firebase Authentication; the application shall never store raw passwords in Firestore or local device storage.
- All Firestore read/write operations shall be authorized through Security Rules verifying authentication state, role, and restaurant membership before granting access.
- All network communication (client ↔ Firebase, client ↔ Gemini proxy) shall use TLS-encrypted HTTPS.
- A Staff account's client-side and server-side permissions shall both deny access to financial data (defense in depth, per FR-008).
- **[NEW]** The Gemini API key shall never be embedded in the client bundle; all AI requests shall route through a server-side Cloud Function proxy.
- **[NEW]** Firestore Security Rules and Cloud Functions shall be covered by automated rule-unit tests before each deployment.
- **[NEW]** Rate limiting shall be applied to authentication attempts (max 5 failed attempts per 15 minutes per account) to mitigate brute-force attacks.

4.3 Reliability

- The system shall achieve ≥99% successful synchronization of locally queued writes (batch entries, waste logs) once connectivity is restored after interruptions of up to 10 minutes.
- The system shall not lose a submitted waste log or batch entry on application crash occurring after local save but before remote sync; the entry shall retry automatically on next launch.

4.4 Availability

- **[NEW]** The system shall target 99.5% monthly uptime for Firebase-hosted services, subject to Firebase's own SLA; the application shall degrade gracefully (read-only cached view) during backend outages rather than crashing.

4.5 Scalability

- The data model and Firestore query structure shall support at least 50 concurrent restaurant accounts and up to 5,000 active batches per restaurant without schema changes.
- The notification dispatch mechanism shall support at least 20 concurrently signed-in devices per restaurant without measurable increase (>30 seconds) in delivery latency.

4.6 Usability

- A new Staff user shall complete batch registration within 60 seconds of opening the relevant screen without external instructions, verified via a usability walkthrough with ≥3 unfamiliar test users.
- Expiry status shall be conveyed through both colour coding and explicit text labels, for users with colour vision deficiency.
- All destructive actions (batch deletion, staff removal, waste void) shall require explicit confirmation before execution.

4.7 Accessibility

- **[NEW]** All interactive elements shall have a minimum touch target of 44×44 dp.
- **[NEW]** Text contrast ratios shall meet WCAG 2.1 AA (minimum 4.5:1 for body text).
- **[NEW]** Screen-reader labels (accessibility labels) shall be provided for all icons and status tags.

4.8 Maintainability

- **[NEW]** The codebase shall follow a modular, layered architecture (Presentation / State / Service / Navigation, per Section 5) to isolate third-party SDK changes.
- **[NEW]** All Cloud Functions shall be covered by unit tests with a minimum 70% coverage target for business-logic modules (cost calculation, expiry evaluation, waste-loss calculation).
- **[NEW]** Code shall be linted (ESLint) and type-checked (TypeScript strict mode) as part of the CI pipeline.

4.9 Compatibility

- The application shall function correctly on Android 8.0 (API 26) through the latest publicly available Android version at time of testing.
- The application shall support screen sizes from 4.7" to 6.9" without truncating or overlapping critical UI elements.
- The application shall function correctly with the device's system language set to English; other languages are out of scope for this version.

4.10 Compliance

- **[NEW]** The system shall handle user PII (email, name) in accordance with general data-protection best practice (data minimization, purpose limitation); no PII shall be sent to the Gemini API beyond what is operationally necessary for the assistant feature.
- **[NEW]** As an academic project, formal regulatory certification (e.g., PCI-DSS, HIPAA) is out of scope; standard secure-development practices are followed voluntarily.

4.11 Backup and Recovery

- **[NEW]** Firestore's built-in point-in-time recovery / daily export (via scheduled Cloud Function to Cloud Storage) shall be configured to allow restoration of restaurant data in case of accidental bulk deletion.
- **[NEW]** Soft-delete (archive) patterns shall be used for batches and waste logs (FR-019, FR-031) instead of hard deletes, to support recovery without a separate backup restore.

4.12 Logging and Monitoring

- **[NEW]** Cloud Function errors (expiry checks, notification dispatch, Gemini proxy failures) shall be logged to Firebase Cloud Logging / Crashlytics with severity levels.
- **[NEW]** Client-side crash reporting shall be integrated (e.g., Firebase Crashlytics) to capture unhandled exceptions with device/OS metadata.
- **[NEW]** Key business metrics (batches created/day, waste logged/day, notification delivery success rate) shall be observable via a lightweight operational dashboard or Firebase Analytics events.

---

5. User Roles and Permissions

Capability	Admin	Staff	Pending Staff
Register restaurant account	✅	❌	❌
Register with restaurant code	❌	✅	—
Approve/reject staff	✅	❌	❌
Deactivate staff	✅	❌	❌
View inventory	✅	✅	❌
Create/edit batch	✅	✅	❌
Archive batch	✅	❌	❌
Log waste	✅	✅	❌
Void waste entry	✅	❌	❌
View cost/expense figures	✅	❌	❌

View analytics dashboard	✅	❌	❌
Export reports	✅	❌	❌
Use AI Assistant (non-financial scope)	✅	✅	❌
Use AI Assistant (financial scope)	✅	❌	❌
View audit log	✅	❌	❌
Edit restaurant settings	✅	❌	❌
Edit own profile	✅	✅	✅ (limited)
Receive expiry notifications	✅	✅	❌

---

6. Use Cases

UC-01: Staff Registers and Awaits Approval

- **Actor:** Prospective Staff user
- **Trigger:** User downloads app, taps "Register as Staff"
- **Flow:** Enter email/password/restaurant code → system validates code → account created with status Pending → user routed to Pending Approval screen → Admin receives no automatic notification in MVP **[ASSUMPTION: Admin must check Staff Management screen; push notification to Admin for new pending requests is a future enhancement]** → Admin approves → Staff gains access on next data refresh.
- **Postcondition:** Staff has operational access.

UC-02: Admin Registers a Batch Delivery

- **Actor:** Admin or approved Staff
- **Flow:** Open Batch Registration → fill ingredient, quantity, unit, cost, supplier, dates → submit → validated → batch appears at top/bottom of FIFO-ordered list per grouping rule.
- **Postcondition:** Inventory updated in real time across devices.

UC-03: Expiry Alert and Response

- **Actor:** System (automated) / Staff
- **Flow:** Scheduled Cloud Function detects batch entering Amber threshold → notification created and dispatched via FCM → Staff device displays push notification → Staff opens app, views flagged batch, prioritizes its use → optionally logs waste if the item is later found spoiled.
- **Postcondition:** Waste, if any, is logged and reflected in cost-loss totals.

UC-04: Admin Reviews Analytics

- **Actor:** Admin
- **Flow:** Open Analytics Dashboard → select date range → view waste-cost trend, top wasted ingredients, cost breakdown, inventory valuation → optionally export report.
- **Postcondition:** Admin makes a purchasing/process decision informed by data.

UC-05: Staff Queries AI Assistant

- **Actor:** Staff
- **Flow:** Open AI Assistant → type "What's expiring this week?" → system builds role-filtered context (inventory only, no cost) → Gemini API returns natural-language summary → displayed to user.
- **Postcondition:** No data mutation occurs; query optionally saved to session history.

UC-06: Admin Deactivates a Staff Member

- **Actor:** Admin
- **Flow:** Open Staff Management → select Staff → Deactivate → confirm → account and all its active sessions lose access within 60 seconds; FCM token removed.
- **Postcondition:** Former Staff member cannot read/write any restaurant data.

---

7. Business Rules

1. **BR-01:** A restaurant code is unique system-wide and immutable once generated.
2. **BR-02:** A user account belongs to exactly one restaurant; cross-restaurant membership is not permitted.
3. **BR-03:** Expiry status thresholds are fixed at Green (>3 days), Amber (0–3 days), Red (<0 days) unless overridden by restaurant-level configuration (FR-025).
4. **BR-04:** Cost-loss for a waste entry is always computed using the unit cost recorded on the source batch at time of receipt, not any current/market price.
5. **BR-05:** Financial data (cost, inventory valuation, waste-loss totals) is visible to Admin accounts only, with no exception.
6. **BR-06:** A batch cannot be edited or deleted once marked "Consumed," except by an Admin performing an audited correction.
7. **BR-07:** The AI Assistant may never write to Firestore; it is strictly read/summarize.
8. **BR-08:** A Staff account must be in "Approved" status to perform any inventory or waste operation.
9. **BR-09:** Duplicate expiry notifications for the same batch/status transition are suppressed within a 24-hour window.
10. **BR-10 [NEW]:** All monetary figures are rounded to 2 decimal places for display but stored at full precision internally.
11. **BR-11 [NEW]:** Voided waste entries do not affect current batch consumption history beyond restoring quantity; they remain visible in the audit trail.

8. Data Requirements

8.1 Main Entities

User

Attribute	Description
userId	Firebase Auth UID (PK)
email	Login identifier
displayName **[NEW]**	User's display name
role	Admin | Staff
restaurantId	FK → Restaurant
status	Pending | Approved | Rejected | Deactivated
fcmToken	Current device token
notificationPrefs **[NEW]**	Object: {amber: bool, red: bool}
createdAt	Timestamp
lastLoginAt **[NEW]**	Timestamp

Restaurant

Attribute	Description
restaurantId	Unique identifier (PK)
name	Display name
restaurantCode	Unique registration code
adminUserId	FK → User (Admin)
currency	Configured currency code
amberThresholdDays **[NEW]**	Default 3

createdAt	Timestamp

InventoryBatch

Attribute	Description
batchId	Unique identifier (PK)
restaurantId	FK → Restaurant
ingredientName	Grouping key
quantity	Remaining quantity
unitOfMeasure	kg, litre, unit, etc.
unitCost	Cost per unit at receipt
supplier	Supplier name
dateReceived	Date
expiryDate	Date
status	Derived: Green | Amber | Red
isConsumed **[NEW]**	Boolean
isArchived **[NEW]**	Boolean
createdBy	FK → User
lastModifiedBy **[NEW]**	FK → User
lastModifiedAt **[NEW]**	Timestamp

WasteLog

Attribute	Description
wasteLogId	Unique identifier (PK)
restaurantId	FK → Restaurant
batchId	FK → InventoryBatch
quantityWasted	Quantity
reason	Expired | Burnt | Prep Waste | Leftovers

costLoss	Calculated monetary loss
loggedBy	FK → User
timestamp	Date/time
isVoided **[NEW]**	Boolean

Notification

Attribute	Description
notificationId	Unique identifier (PK)
restaurantId	FK → Restaurant
batchId	FK → InventoryBatch
type	ExpiryAmber | ExpiryRed
message	Rendered text
sentAt	Timestamp
recipientDeviceTokens	List of FCM tokens
readBy **[NEW]**	List of userIds who marked read

AuditLog [NEW]

Attribute	Description
auditId	Unique identifier (PK)
restaurantId	FK → Restaurant
actorUserId	FK → User
actionType	Create | Update | Delete | Approve | Reject | Deactivate | Void
entityType	Batch | WasteLog | User | Restaurant
entityId	Target document ID
beforeValue / afterValue	JSON snapshot (where applicable)
timestamp	Date/time

8.2 Entity Relationships

- One Restaurant → many Users (one Admin, many Staff); each User belongs to exactly one Restaurant.
- One Restaurant → many InventoryBatches; each Batch belongs to one Restaurant, created by one User.
- One InventoryBatch → zero or many WasteLog entries; each WasteLog references one Batch and one User.
- One InventoryBatch → zero or many Notification records; each Notification references one Batch and one Restaurant.
- One Restaurant → many AuditLog entries; each entry references one actor User.

8.3 Data Validation Rules

- `quantity`, `unitCost`, `quantityWasted` must be non-negative numerics; `quantity`/`unitCost` on batch creation must be > 0 / ≥ 0 respectively.
- `expiryDate` ≥ `dateReceived`.
- `email` must match RFC 5322 format; enforced client-side and by Firebase Authentication.
- `restaurantCode` must match an existing Restaurant document exactly (case-insensitive comparison recommended **[ASSUMPTION]**).
- `reason` on WasteLog must be one of the four enumerated values only.
- All monetary fields stored as integers in minor currency units (e.g., cents) **[ASSUMPTION]** to avoid floating-point rounding errors, converted to display format client-side.

---

9. External Interface Requirements

9.1 User Interface

Key screens: Login/Registration (with restaurant-code entry for Staff), Pending Approval, Staff Management (Admin), Inventory List (FIFO-grouped, colour-coded), Batch Registration form, Waste Logging form, Analytics Dashboard (Admin), AI Assistant chat, Notifications inbox, **[NEW]** Profile/Settings screen, **[NEW]** Audit Log viewer (Admin).

Navigation flow: Post-authentication routing is role- and status-aware: Pending Staff → Pending Approval screen (held until approved); Approved Staff → Inventory List with access to Batch Registration, Waste Logging, expiry views, and Notifications, with no route to Staff Management or Analytics; Admin → Dashboard home with full navigation to Inventory, Waste Logging, Staff Management, Analytics, AI Assistant, Settings, and Audit Log.

9.2 APIs

- **Firebase Authentication SDK:** email/password sign-up, sign-in, session token issuance, password reset.
- **Firestore SDK:** real-time CRUD for all collections in Section 8, governed by Security Rules.
- **Firebase Cloud Messaging:** HTTPS-based push delivery to registered device tokens.
- **Cloud Functions (HTTPS callable) [NEW]:**
    - `checkExpiryAndNotify` — scheduled function (e.g., every 15 minutes) evaluating batch status and dispatching FCM notifications.
    - `geminiProxy` — callable function accepting {query, role, restaurantId}, constructing role-filtered context, and forwarding to Gemini API; returns natural-language response within a 15-second timeout budget.
    - `computeCostLoss` — server-side validation/derivation of `costLoss` on waste log writes, to prevent client-side tampering (supports FR-028).
- **Google Gemini API:** HTTPS REST interface accepting a context payload and natural-language query, returning a natural-language response; accessed only via `geminiProxy`.

9.3 Database

Firebase Firestore (NoSQL document database). Collections are scoped either as subcollections of `/restaurants/{restaurantId}/` or as top-level collections carrying a `restaurantId` field, with Security Rules enforcing restaurant-scoped, role-based access on every read/write.

9.4 Third-Party Integrations

- Firebase Authentication, Firestore, Cloud Messaging, Cloud Functions (Google/Firebase)
- Google Gemini API (AI natural-language insight generation)
- Victory Native (client-side charting library)
- **[NEW]** Firebase Crashlytics / Cloud Logging (monitoring)
- **[NEW]** Firebase Cloud Storage (for scheduled data export/backup)

9.5 Hardware Interfaces

- Touchscreen Android device with network connectivity (Wi-Fi or cellular), Android 8.0+.
- Device notification system (via FCM and Android OS notification manager) for expiry alerts; no camera, GPS, NFC, or other sensors required.

---

10. System Constraints

- Firebase-only backend; no custom server infrastructure permitted within project scope.
- Android-only client; no iOS or web client deliverable expected for this course submission (React Native's cross-platform capability is available but not required to be exercised).
- Single-currency, single-language (English) system for this version.
- Free-tier service quotas (Firebase Spark plan, Gemini free tier) bound the system's real-world concurrent-user ceiling; production scale-up beyond course demonstration volumes would require paid tier upgrades.
- Team of three developers with an academic timeline; scope is calibrated accordingly (see Section 12 for deferred features).

---

11. Risks and Assumptions

11.1 Risks

Risk	Impact	Likelihood	Mitigation
Gemini API free-tier quota exhausted during demo/testing	Medium	Medium	Cache recent AI responses; implement request throttling; fallback error message (FR-046)
Firestore Security Rules misconfigured, exposing financial data to Staff	High	Low–Medium	Automated rule-unit tests (NFR 4.2); manual penetration-style review before submission
Scheduled Cloud Function fails silently, delaying expiry alerts	Medium	Medium	Cloud Logging alerts on function failure (NFR 4.12); manual "recalculate now" fallback on inventory open (FR-016)
Team member availability/skill gaps (small 3-person team) affect delivery of AI/analytics modules	Medium	Medium	Prioritize High-priority FRs first; defer Low-priority items (export, audit UI) if timeline tightens
Floating-point rounding errors in cost calculations	Low–Medium	Medium	Store monetary values in minor units (cents) per data validation rule in 8.3

11.2 Assumptions (Consolidated)

- Single Admin per restaurant in MVP.
- No payment processing or subscription billing required.
- Gemini API access routed through a Cloud Function proxy rather than direct client calls.
- Restaurant code comparison is case-insensitive.
- Monetary values stored in minor units (cents) internally.
- No formal regulatory compliance certification required (academic scope).
- APK/Expo sideload distribution is sufficient; app-store publication not required.

---

12. Future Enhancements

- Multi-admin / multi-manager support per restaurant
- Multi-restaurant management under a single account (restaurant chains)
- Supplier ordering / procurement automation and reorder-point suggestions
- Point-of-sale (POS) integration for automatic ingredient consumption tracking
- Offline-first conflict resolution beyond Firestore's default caching
- Multi-currency and multi-language (localization) support
- Barcode/QR-code scanning for faster batch entry
- Predictive waste analytics using historical trend modeling (beyond descriptive dashboard)
- Push notification to Admin when a new Staff registration is pending (reduces manual check-in, referenced in UC-01)
- Email digest reports (weekly waste/cost summary) in addition to in-app export
- Role granularity beyond Admin/Staff (e.g., "Manager" with partial financial visibility)

End of Document — Restora SRS v2.0 (Consolidated & Enhanced)